Linux System Administration (E1ITA307)

Linux Infrastructure Deployment Project Report

On

ZERO-TRUST ENTERPRISE STORAGE VAULT WITH AUTOMATED RANSOMWARE DEFENSE AND DISASTER ROLLBACK

Submitted by

DHARSHINI K (2411021060912)

KANI V (2411021061023)

HEMANATH K (2411021060862)

IBADUL AZAM (2411021060840)

In partial fulfillment of the award of the degree of

BACHELOR OF TECHNOLOGY

In

Information Technology

Under the Supervision of

Prof. Kusuma J

ALLIANCE SCHOOL OF ADVANCED COMPUTING

ALLIANCE UNIVERSITY

BENGALURU

Oct 2026

DEPARTMENT OF INFORMATION TECHNOLOGY

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the Project work entitled **"ZERO-TRUST ENTERPRISE STORAGE VAULT WITH AUTOMATED RANSOMWARE DEFENSE AND DISASTER ROLLBACK"** submitted by Dharshini K [2411021060912], Kani V [2411021061023], Hemanath K [2411021060862] and Ibadul Azam [2411021060840] in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Alliance University, is a bonafide work during the academic year 2026-2027.

Prof. Kusuma J	Prof. Ananthanagu U	Dr. Shekhar R
Course Handling Faculty	Course Coordinator	Professor & HoD

TABLE OF CONTENTS

1. Introduction 4

1.4 Objectives and Scope 5

2. Methodology 6

3.1 System Architecture 7

3.2 Linux Environment Setup 8

3.3 User and Group Management 9

3.4 File and Directory Management 10

3.5 Package and Process Management 11

3.6 Storage and File System Configuration 12

3.7 Backup and Automation 13

3.8 Security and Access Control 15

3.9 Terminal Menu and Error Handling 17

4. Results and Testing 18

5. Conclusion and Future Scope 19

References 20

Appendix A: Setup Commands 21

Appendix B: Code Explanation and Demonstration 23

Appendix C: Complete Bash Source 25

Appendix D: Implementation Evidence 33

1. INTRODUCTION

1.1 Aim

The aim is to implement a menu-driven storage vault using Bash shell scripting and Linux commands. The application demonstrates restricted user access, a protected clean backup, SHA-256 canary verification, scheduled security checks, automatic access blocking, and administrator-selected recovery.

1.2 Background

Important files may be changed or deleted accidentally or by malicious software. File permissions reduce unauthorized access, monitoring can reveal selected suspicious changes, and verified backups support recovery. This project demonstrates these concepts with fictional student documents so that each operation is easy to explain.

The term zero trust is used here to describe least-privilege access and repeated integrity checking. This is an educational prototype, not a complete enterprise zero-trust platform or a general ransomware detector.

1.3 Problem Statement

A student needs to edit an assignment while outsiders must be denied access. The student must also be unable to alter the protected backups or trusted fingerprints. When a designated trap file changes, the application must record the incident and restrict student access. The administrator must be able to verify and restore the original files.

The terminal interface accepts menu choices and text input, runs Bash functions, displays results, and handles invalid input. No Python backend, browser interface, or inotify-based implementation is used in this version.

1.4 OBJECTIVES AND DEMONSTRATION SCOPE

1. Store assignment.txt, marks.txt and canary.txt in a controlled vault.

2. Separate the authorized student account from an outsider test account.

3. Accept a sentence from the user and append it to the assignment.

4. Create one protected clean backup and trusted SHA-256 fingerprints.

5. Compare the canary with its baseline during manual or scheduled checks.

6. Record an incident and block student vault access if the check fails.

7. Verify the backup and recovered files before reopening the vault.

8. Demonstrate invalid menu choices, empty input and access errors.

Simple example used in the demonstration

assignment.txt contains sample Linux coursework. marks.txt contains fictional marks. canary.txt is a trap file that normal work should not change. A normal assignment edit is allowed; the canary simulation intentionally changes only the sample trap file.

Scope of automation

A systemd timer checks the canary approximately every ten seconds while Ubuntu is running. Detection and access blocking are automatic when that timer is enabled. Recovery is manually selected by the administrator and requires the word RESTORE. Backups are also created through the menu, not automatically scheduled.

2. METHODOLOGY

2.1 Tools and Technologies Used

Ubuntu Linux and Bash provide the operating environment, functions, terminal menu and input processing.

Linux account and permission tools include groupadd, useradd, runuser, chmod and install.

sha256sum records and checks digital fingerprints; tar creates, compares and extracts compressed backups.

systemd schedules canary checks; flock prevents overlapping menu and monitor operations.

cat, printf, tee, tail and date support input/output and timestamped logs.

2.2 Development Process

The environment was inspected, project accounts and directories were created, and sample student files were added. Access tests verified the permissions. The Bash menu then combined normal input, backup verification, canary checking, automatic containment, logging and manual recovery.

2.3 Testing and Evidence

The demonstration used harmless sample data. Nineteen terminal screenshots record menu interaction, authorization tests, an existing backup check, timer activation, the canary simulation, automatic blocking, recovery and input errors. Each screenshot is placed with its matching caption in Appendix D; page 18 summarizes the observed results.

3.1 SYSTEM ARCHITECTURE

The administrator uses the terminal menu, while the systemd timer independently invokes the monitor. Both enter the Bash script and share an operation lock so that monitoring does not overlap a menu operation.

Student file operations use runuser. Backups, baselines, incident state and logs are restricted to root. A failed canary check blocks student access; recovery is selected by the administrator.

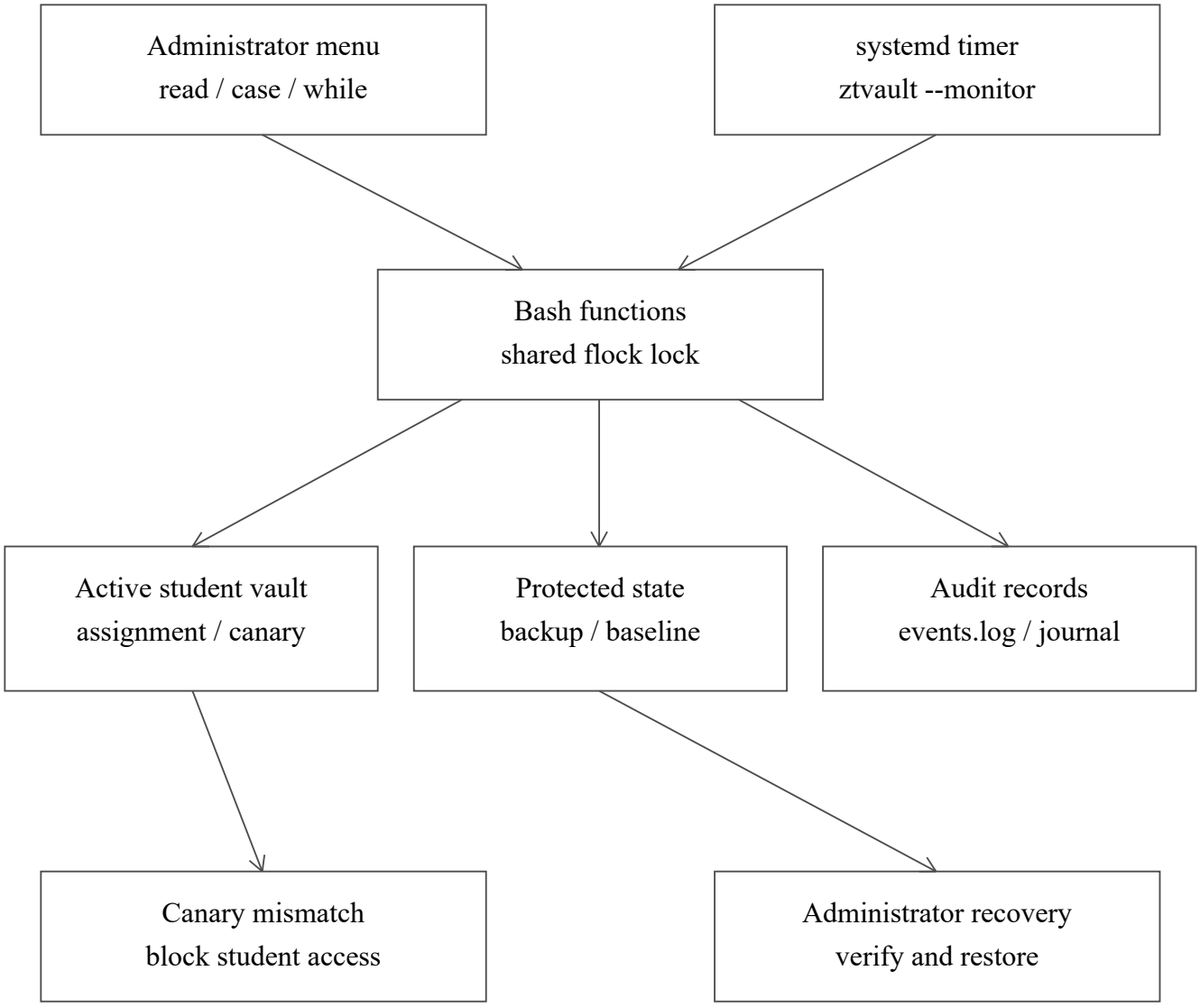


Figure 3.1. Components and control paths of the Bash vault.

3.2 LINUX ENVIRONMENT AND PROJECT SETUP

Observed environment

The supplied environment output identifies Ubuntu 26.04 LTS on WSL2, kernel 6.18.33.1-microsoft-standard-WSL2, with systemd as process 1. The Ubuntu root filesystem is ext4. Bash, Python, tar, sha256sum, flock, systemctl and sudo were available; Python is not used by this implementation.

Project locations

/usr/local/sbin/ztvault - installed Bash application.

/srv/ztvault/data - active student documents.

/var/lib/ztvault/backups - clean archive and archive checksum.

/var/lib/ztvault/baselines - original-file and canary fingerprints.

/var/lib/ztvault/recovery - staged and previous file copies.

/var/log/ztvault/events.log - event log.

Starting the application

sudo bash -n /usr/local/sbin/ztvault
sudo ztvault

bash -n checks syntax without running the script. sudo ztvault starts the administrative terminal menu. The timer runs only while the Ubuntu instance is running; shutting down WSL stops its monitoring.

3.3 USER AND ACCESS CONTROL

The authorized demonstration user is ztv_writer, a member of ztv_access. The unauthorized demonstration user is ztv_outsider. Both accounts have interactive login disabled. The administrator uses runuser to perform controlled operations as these accounts.

Normal access permissions

The root-owned active directory uses group ztv_access and mode 2770. Its set-group-ID bit makes newly created entries inherit the directory group. The three initial sample files use ownership root:ztv_access and mode 0660. Protected state and log directories use root ownership and mode 0700.

Permission checks

```
sudo runuser -u ztv_writer -- \
  cat /srv/ztvault/data/assignment.txt
sudo runuser -u ztv_outsider -- \
  cat /srv/ztvault/data/assignment.txt
```

The first command should display the assignment. The outsider command should return Permission denied. A separate test verifies that the student cannot list /var/lib/ztvault/backups.

Access during an incident

The monitor changes the active directory to mode 0700 and writes an incident marker. This removes student directory access and prevents new path-based reads or writes through that directory. The application remains locked until recovery completes. This does not revoke file descriptors that a process already has open.

3.4 FILE AND DIRECTORY MANAGEMENT

3.4.1 Directory structure and sample data

The active documents are stored in /srv/ztvault/data. The application separates this student-accessible directory from administrator-only backups, baselines and recovery files under /var/lib/ztvault. Logs are stored separately in /var/log/ztvault.

assignment.txt contains sample coursework, marks.txt contains fictional marks and canary.txt is the security trap file. The menu can display the assignment and marks and append one validated sentence to the assignment.

3.4.2 Ownership, permissions and symbolic links

The initial documents are owned by root:ztv_access with mode 0660. The directory uses mode 2770 during normal operation. The application checks that the three sample paths are regular files and rejects symbolic links in its sample-file check. It also explicitly rejects a symbolic-link canary during monitoring. No symbolic-link feature is required by this prototype.

3.4.3 Searching and log filtering

```
sudo find /srv/ztvault -maxdepth 2 -type f
sudo ls -ld /srv/ztvault/data
sudo tail -n 25 /var/log/ztvault/events.log
sudo grep ALERT /var/log/ztvault/events.log
```

find lists files, ls shows permissions, tail displays recent events and grep filters alert entries. The log may not exist until an operation has recorded an event. These commands support inspection; they are not additional claimed security mechanisms.

3.5 PACKAGE AND PROCESS MANAGEMENT

3.5.1 Packages and dependencies

The observed environment already provides the commands needed for this Bash implementation. No Flask, Python package, browser dashboard or inotify-tools installation is required. APT is Ubuntu's package manager, but package installation or updates are not claimed as completed project operations.

```
command -v bash tar sha256sum flock systemctl runuser
dpkg-query -W bash coreutils tar util-linux systemd
```

command -v checks command availability. dpkg-query reports installed package versions. If a dependency is missing on another computer, install the corresponding Ubuntu package before the demonstration.

3.5.2 Process and resource inspection

```
ps -p 1 -o comm=
ps -eo pid,user,comm --sort=pid | head -n 15
free -h
```

These commands identify the startup system, show running processes and display memory use. WSL2 reports systemd as process 1 in the supplied environment. The script uses short command executions rather than a permanent application server.

3.5.3 Service management

```
systemctl status ztvault-monitor.timer --no-pager
sudo journalctl -u ztvault-monitor.service -n 15 --no-pager
```

Use these after menu option 10 has installed the units. The oneshot service can be inactive between checks even while its timer is active; the timer status and journal provide the relevant scheduling evidence.

3.6 STORAGE AND FILE SYSTEM CONFIGURATION

3.6.1 Existing filesystem

The project uses the existing ext4 filesystem inside the Ubuntu WSL2 instance. It does not format a disk, repartition storage, create an LVM volume or require mounting another filesystem. File-level permissions protect the demonstration paths.

```
findmnt -T /srv/ztvault/data
df -h /srv/ztvault/data
df -i /srv/ztvault/data
lsblk -f
```

findmnt identifies the containing mount. df -h reports filesystem space and df -i reports inode usage. lsblk -f displays available block-device information. These are environment inspection commands, not scheduled disk-threshold monitoring in ztvault.

3.6.2 Storage allocation

```
sudo du -sh /srv/ztvault /var/lib/ztvault /var/log/ztvault
```

du reports the space occupied by the active vault, protected backup/recovery state and logs. The files are small text examples, but preserved recovery directories can accumulate after repeated demonstrations.

3.6.3 Storage limitations

The backup is on the same Linux filesystem as the active documents. Student permissions protect it from the ordinary project account, but this is not an immutable, offline or physically separate backup. Complete disk failure or privileged compromise can affect both. Off-device backups remain future work.

3.7 BACKUP AND DISASTER RECOVERY

Clean backup creation

Option 4 creates clean-backup.tar.gz containing the three sample files. Student directory access is temporarily restricted during creation. The script records file fingerprints, compares the archive with the source files, checks the original hashes and saves an archive checksum. It preserves an existing clean backup instead of replacing it with potentially changed files.

Backup verification

```
cd /var/lib/ztvault/backups
sha256sum --check clean-backup.sha256
```

A matching checksum detects no change relative to the stored archive fingerprint. Because both are on the same computer, this is not protection against an administrator compromising both files or against complete disk loss.

Administrator-selected recovery

Option 8 first checks that the backup and baselines exist and verifies the archive checksum. After the administrator types RESTORE, it extracts into a temporary recovery directory and checks both the original-file and canary fingerprints.

After successful staging, the current vault is locked and moved into a protected previous-files directory. The script recreates the active vault, installs the three verified documents, checks their hashes again, restores mode 2770 and removes the incident marker.

Recovery returns the documents to the clean-backup state. Later legitimate edits will therefore disappear from the active files, but the previous vault is preserved for inspection. This is manual recovery after automatic detection and containment, not automatic rollback.

3.7 BACKUP AND AUTOMATION - CONTINUED

3.7.1 Backup strategy and frequency

Menu option 4 creates one clean backup, verifies it and preserves it. Re-selecting the option verifies the existing clean archive. There is no recurring backup schedule and no multiple-version backup selector. Files created outside the three selected sample documents are not included in the clean archive.

3.7.2 Scheduling mechanism

The reference project uses cron; this project uses a systemd timer. Both are Linux scheduling mechanisms, but the implementation and evidence must match systemd here. The timer runs the Bash monitor approximately every ten seconds while the Ubuntu instance is running.

systemctl list-timers --all ztvault-monitor.timer
systemctl cat ztvault-monitor.service
systemctl cat ztvault-monitor.timer

3.7.3 Concurrency and logs

flock serializes menu operations against the monitor. If a menu operation holds the lock, an automatic check skips that run. Checks resume on later timer activations. UPDATE, BACKUP, DEMO, ALERT, AUTOMATION and RECOVERY events are timestamped in events.log. Normal SAFE messages from the scheduled service are available in the journal.

3.7.4 Stop and restart the timer

sudo systemctl stop ztvault-monitor.timer
sudo systemctl start ztvault-monitor.timer

Use stop only when monitoring should be paused; restart before demonstrating automatic detection. Exiting the terminal menu does not itself stop an enabled timer.

3.8 SECURITY AND ACCESS CONTROL

Trusted fingerprints

Backup creation saves original-files.sha256 for the three sample files and canary.sha256 for the trap file. These files are stored under the administrator-only baseline directory. The monitor compares the current canary against this saved fingerprint.

```
cd /srv/ztvault/data
sha256sum --check --status \
  /var/lib/ztvault/baselines/canary.sha256
```

Detection and containment

If the canary is a regular file, is not a symbolic link and matches its fingerprint, the monitor reports SAFE. A failed check, missing canary or invalid canary causes the application to restrict vault access, create an incident marker and record an ALERT in the event log. A missing baseline produces an error because monitoring has not been initialized.

Automatic execution

Menu option 10 creates ztvault-monitor.service and ztvault-monitor.timer. The service runs /usr/local/sbin/ztvault --monitor. The timer uses OnActiveSec=10s, OnUnitActiveSec=10s and AccuracySec=1s. Checks can be delayed while another operation holds the shared flock lock.

Safe simulation

Option 6 asks the user to type DEMO and appends harmless text to canary.txt as the student. It does not encrypt files or run malware. The timer should detect the mismatch; option 5 can also check immediately. Option 7 verifies that student writing is denied after containment.

3.8 SECURITY CONTROLS - CONTINUED

3.8.1 Least privilege and administrator boundary

The student and outsider are dedicated test identities. Interactive login is disabled with /usr/sbin/nologin. The administrator starts the menu through sudo; student operations explicitly use runuser. No custom password authentication, multi-factor authentication or identity provider is implemented.

3.8.2 Baselines and controlled input

Trusted fingerprints and the backup are stored in root-only directories. Text input is quoted and passed as data through printf and tee, rather than evaluated as a command. The menu rejects invalid options; assignment input must be nonblank and no longer than 200 characters.

3.8.3 Network security scope

The project operates locally and does not open a web server or network listener. Firewall rules and SSH services are not configured by the application. Their presence, absence or status must not be inferred from the reference report or claimed as tested on this computer.

3.8.4 What containment means

Mode 0700 on the root-owned vault directory blocks new student path-based access. It does not terminate malicious processes, stop root, undo prior changes or revoke already-open file descriptors. Canary checks also cannot identify attacks that leave the trap file unchanged.

3.8.5 Recovery authorization

The administrator selects recovery and types RESTORE. Recovery validates the clean copy before replacing the active sample files and preserves the previous directory. This is verified manual rollback after optional automatic detection and containment.

3.9 TERMINAL MENU AND ERROR HANDLING

1. View assignment and marks.

2. Add a sentence to the assignment.

3. Test access permissions.

4. Create or verify the clean backup.

5. Check security now.

6. Simulate a suspicious canary change.

7. Test write blocking after an incident.

8. Recover original files.

9. View the security log.

10. Enable automatic monitoring.

0. Exit.

User interaction and processing

read -r captures the menu choice or sentence. case selects the relevant function. The update function rejects blank input and input longer than 200 characters. Accepted text is passed to tee as data and is not evaluated as a shell command.

Error handling

Unknown choices display a range error and keep the menu running. Missing files, unavailable backups and absent baselines produce errors. DEMO and RESTORE must match exactly; other input cancels the action. Failed operations report the error and return to the menu. Administrative operations require root.

4. RESULTS AND TESTING

4.1 Observed demonstration results

Nineteen supplied terminal screenshots document the complete demonstration on Ubuntu WSL2. The results below are supported by the corresponding figures in Appendix D. The earlier setup transcript also confirmed authorized writing and access restrictions.

4.2 Test record

Environment, users and permissions: PASS - D1 and D2.

Menu and input saving: PASS - D3 and D4.

Authorized and unauthorized access tests: PASS - D5.

Existing clean backup checksum verification: PASS - D6.

Normal assignment edit leaves canary SAFE: PASS - D7.

Timer enabled and active: PASS - D8.

Safe simulation and automatic containment: PASS - D9 and D10.

Timestamped incident logging: PASS - D11.

Verified manual recovery and restored original: PASS - D12 and D13.

Invalid choice and empty input rejected: PASS - D14.

4.3 Interpretation of results

The log records a simulated canary change at 13:37:30 and an alert at 13:37:34; the service journal corroborates automatic containment. Four seconds is the observed delay for this run, not a guaranteed response time. Recovery was explicitly selected by the administrator. Backup evidence shows verification of an existing archive.

5. CONCLUSION AND FUTURE SCOPE

Conclusion

The project demonstrates a Bash-based student-file vault with a terminal interface, restricted accounts, a protected clean backup, trusted SHA-256 fingerprints, canary checks, logging and recovery. The screenshots show successful input handling, access restrictions and backup verification.

The active systemd timer detected a deliberate canary change and automatically blocked student access. The service journal and security log record the incident. Administrator-selected recovery verified the backup and restored the original files, after which the canary check returned SAFE. Invalid choices and empty input were rejected.

Limitations

An attack that leaves the canary unchanged may not be detected.

Directory permission changes do not stop root or revoke existing open file handles.

Monitoring runs only while Ubuntu is running and may skip a check while another operation holds the lock.

The clean backup is local and single-version. It is not an offline or immutable backup.

This is an educational demonstration of selected zero-trust principles, not a production enterprise platform.

Future Scope

Extensions could include protected off-device backups, multiple verified backup versions, broader integrity policies, stronger process containment and authenticated administrative roles.

REFERENCES

The following Linux manual pages document the commands and service concepts used in the implementation:

1. Bash shell: man bash.

2. GNU Coreutils: man sha256sum, man chmod, man install.

3. GNU tar: man tar.

4. Account management: man useradd, man groupadd, man runuser.

5. File locking: man flock.

6. Scheduling: man systemd.timer and man systemd.service.

7. Service management: man systemctl and man journalctl.

Implementation and evidence

Application: /usr/local/sbin/ztvault. Appendix C contains the Bash source listing. Appendix D contains nineteen user-supplied terminal screenshots from the project demonstration on 23 October 2026. The environment and setup access-test transcript provide additional verification.

APPENDIX A: INITIAL SETUP COMMANDS

These are fresh-install commands explaining the setup already completed. Do not re-run account creation on an existing installation. The sample accounts have no interactive login. The commands must be executed by an administrator.

```
sudo groupadd ztv_access
sudo useradd -M -g ztv_access \
  -s /usr/sbin/nologin ztv_writer
sudo useradd -M -U -s /usr/sbin/nologin ztv_outsider
sudo install -d -o root -g root -m 0755 /srv/ztvault
sudo install -d -o root -g ztv_access -m 2770 \
  /srv/ztvault/data
sudo install -d -o root -g root -m 0700 \
  /var/lib/ztvault /var/lib/ztvault/backups \
  /var/lib/ztvault/baselines /var/lib/ztvault/recovery \
  /var/log/ztvault
```

Verify accounts and permissions

```
id ztv_writer
id ztv_outsider
sudo ls -ld /srv/ztvault/data /var/lib/ztvault
getent passwd ztv_writer ztv_outsider
```

groupadd creates the access group; useradd creates the identities. install -d creates directories with the specified owner, group and permissions. id and getent verify account details. The existing access-test transcript confirms the initial read, write and denial checks.

APPENDIX A: SAMPLE DATA AND INSTALLATION

For a fresh installation only, create the three example files below. On the existing project, use the recovery menu instead of replacing files or rebuilding the trusted baseline.

```
sudo bash <<'SETUP'
cat > /srv/ztvault/data/assignment.txt <<'EOF'
STUDENT ASSIGNMENT
Student: Demo Student
Subject: Linux System Administration
Topic: File Permissions
EOF
cat > /srv/ztvault/data/marks.txt <<'EOF'
SAMPLE MARKS
Linux: 90
Python: 95
Mathematics: 88
EOF
cat > /srv/ztvault/data/canary.txt <<'EOF'
SECURITY TRAP FILE
Do not edit this file during normal work.
EOF
chown root:ztv_access /srv/ztvault/data/*.txt
chmod 0660 /srv/ztvault/data/*.txt
SETUP
```

Install a saved Bash source file

```
bash -n "$HOME/ztvault.sh"
sudo install -o root -g root -m 0750 \
  "$HOME/ztvault.sh" /usr/local/sbin/ztvault
sudo ztvault
```

The exact wording of the sample documents may differ from your earlier setup; checksums must be created from the clean files actually used. Appendix C provides the menu source listing. Use the existing installed script for the current demonstration.

APPENDIX B: CODE EXPLANATION

A.1 Source file submission

Submit a copy of the actual installed script alongside the report. These commands export a readable copy and check its syntax:

```
sudo cat /usr/local/sbin/ztvault > "$HOME/ztvault.sh"
bash -n "$HOME/ztvault.sh"
```

A.2 Find the relevant code during the review

```
sudo sed -n '/^update_assignment()/,/^}/p' \
  /usr/local/sbin/ztvault
sudo sed -n '/^monitor()/,/^}/p' /usr/local/sbin/ztvault
sudo sed -n '/^recover_files()/,/^}/p' \
  /usr/local/sbin/ztvault
```

A.3 Explain execution step by step

1. Set the project paths and require administrator privileges.

2. Check required directories and open the shared operation lock.

3. Display current status and the menu; read the selected choice.

4. Use case to select the matching Bash function.

5. Lock the operation, validate inputs and run the Linux commands.

6. Print the result or error, release the lock and return to the menu.

7. In --monitor mode, perform one security check without showing the menu.

APPENDIX B: DEMONSTRATION GUIDE

1. Start sudo ztvault and choose 1 to view the documents.

2. Choose 3 to show access restrictions.

3. Choose 4 to create or verify the clean backup.

4. Choose 2 and enter: I completed my Linux assignment.

5. Choose 1 to show the saved input, then 5 for SAFE.

6. Choose 10 to enable automatic monitoring.

7. Choose 6 and type DEMO. Return to the menu and wait about 15 seconds.

8. Choose 7 to show write blocking and 9 to inspect the alert.

9. Choose 8, type RESTORE and capture checksum verification.

10. Choose 1 for the restored assignment and 5 for SAFE.

11. Enter 99; then choose 2 and submit an empty sentence.

Evidence and source submission

Appendix D on pages 33-41 contains nineteen terminal screenshots grouped into fourteen figures. Page 18 records their observed results. Appendix C contains the Bash source listing; submit the executable script alongside the report for the working demonstration.

APPENDIX C: BASH SOURCE - 1 OF 8

Bash source listing for ztvault.

```
#!/bin/bash
set -Eeuo pipefail
PATH=/usr/sbin:/usr/bin:/sbin:/bin
export PATH
umask 077
VAULT=/srv/ztvault/data
BASE=/var/lib/ztvault
BACKUP=$BASE/backups/clean-backup.tar.gz
MANIFEST=$BASE/baselines/original-files.sha256
CANARY=$BASE/baselines/canary.sha256
INCIDENT=$BASE/incident
LOG=/var/log/ztvault/events.log
if (( EUID != 0 )); then
   echo "Please run: sudo ztvault"
   exit 1
fi
for directory in "$VAULT" "$BASE/backups" \
   "$BASE/baselines" "$BASE/recovery" /var/log/ztvault; do
   if [[ ! -d "$directory" || -L "$directory" ]]; then
      echo "ERROR: Required project directory missing: $directory"
      exit 1
   fi
done
exec 9>"$BASE/operation.lock"
log_event() {
   printf '%s | %s\n' "$(date '+%F %T')" "$*" >> "$LOG"
}
require_open() {
   if [[ -f "$INCIDENT" ]]; then
      echo "BLOCKED: An incident is active. Recover the files first."
      return 1
   fi
}
check_samples() {
   local name
   for name in assignment.txt marks.txt canary.txt; do
      if [[ ! -f "$VAULT/$name" || -L "$VAULT/$name" ]]; then
         echo "ERROR: $name is missing or is not a regular file."
         return 1
      fi
   done
}
view_files() {
   require_open || return 0
   check_samples || return 0
   echo "----- ASSIGNMENT -----"
   runuser -u ztv_writer -- cat "$VAULT/assignment.txt"
```

APPENDIX C: BASH SOURCE - 2 OF 8

Bash source listing for ztvault.

```
    echo
    echo "----- SAMPLE MARKS -----"
    runuser -u ztv_writer -- cat "$VAULT/marks.txt"
}
update_assignment() {
    require_open || return 0
    check_samples || return 0
    local note
    read -r -p "Enter one sentence to add to the assignment: " note
    if [[ -z "${note//[[:space:]]/}" ]]; then
        echo "ERROR: Input cannot be empty."
        return 0
    fi
    if (( ${#note} > 200 )); then
        echo "ERROR: Please enter 200 characters or fewer."
        return 0
    fi
    if printf '%s\n' "$note" |
        runuser -u ztv_writer -- tee -a "$VAULT/assignment.txt" \
        >/dev/null; then
        echo "SUCCESS: Assignment updated."
        log_event "UPDATE: Student added a sentence to the assignment."
    else
        echo "ERROR: Could not update the assignment."
    fi
}
test_permissions() {
    require_open || return 0
    check_samples || return 0
    echo "----- STUDENT READ TEST -----"
    if runuser -u ztv_writer -- cat "$VAULT/assignment.txt"; then
        echo "PASS: Student can read the assignment."
    else
        echo "FAIL: Student could not read the assignment."
    fi
    echo "----- OUTSIDER READ TEST -----"
    if runuser -u ztv_outsider -- cat "$VAULT/assignment.txt"; then
        echo "FAIL: Outsider could access the assignment."
    else
        echo "PASS: Outsider access denied."
    fi
    echo "----- STUDENT BACKUP ACCESS TEST -----"
    if runuser -u ztv_writer -- ls "$BASE/backups"; then
        echo "FAIL: Student could access the backup directory."
    else
        echo "PASS: Protected backup access denied."
    fi
```

APPENDIX C: BASH SOURCE - 3 OF 8

Bash source listing for ztvault.

```
}
verify_backup() {
  if [[ ! -f "$BACKUP" ||
        ! -f "$BASE/backups/clean-backup.sha256" ||
        ! -f "$MANIFEST" || ! -f "$CANARY" ]]; then
    echo "ERROR: Complete clean backup is unavailable."
    return 1
  fi
  (cd "$BASE/backups"; sha256sum --check clean-backup.sha256)
}
create_backup() {
  require_open || return 0
  check_samples || return 0
  if [[ -e "$BACKUP" || -e "$MANIFEST" || -e "$CANARY" ||
        -e "$BASE/backups/clean-backup.sha256" ]]; then
    echo "A backup or baseline already exists."
    if verify_backup; then
      echo "Existing backup verified. Original backup preserved."
    else
      echo "ERROR: Existing backup is incomplete or damaged."
    fi
    return 0
  fi
  (
    local work
    work=$(mktemp -d "$BASE/backups/.build-XXXXXX")
    trap 'chmod 2770 "$VAULT"; rm -rf -- "$work"' EXIT
    chmod 0700 "$VAULT"
    cd "$VAULT"
    sha256sum assignment.txt marks.txt canary.txt \
      > "$work/original-files.sha256"
    sha256sum canary.txt > "$work/canary.sha256"
    tar -czf "$work/clean-backup.tar.gz" \
      assignment.txt marks.txt canary.txt
    tar -dzf "$work/clean-backup.tar.gz"
    sha256sum --check "$work/original-files.sha256"
    (cd "$work"; sha256sum clean-backup.tar.gz > clean-backup.sha256)
    install -m 0600 "$work/original-files.sha256" "$MANIFEST"
    install -m 0600 "$work/canary.sha256" "$CANARY"
    install -m 0600 "$work/clean-backup.tar.gz" "$BACKUP"
    install -m 0600 "$work/clean-backup.sha256" \
      "$BASE/backups/clean-backup.sha256"
    log_event "BACKUP: Clean backup and fingerprints created."
    echo "SUCCESS: Clean backup created and verified."
  )
}
monitor() {
```

APPENDIX C: BASH SOURCE - 4 OF 8

Bash source listing for ztvault.

```
    if [[ -f "$INCIDENT" ]]; then
        chmod 0700 "$VAULT"
        echo "LOCKED: Incident active. Recovery is required."
        return 0
    fi
    if [[ ! -s "$CANARY" ]]; then
        echo "ERROR: Create the clean backup before monitoring."
        return 1
    fi
    if [[ -f "$VAULT/canary.txt" && ! -L "$VAULT/canary.txt" ]] &&
        (cd "$VAULT" && sha256sum --check --status "$CANARY"); then
        echo "SAFE: Canary matches its original fingerprint."
    else
        chmod 0700 "$VAULT"
        printf '%s\n' "Canary changed, disappeared, or became invalid." \
            > "$INCIDENT"
        log_event "ALERT: Canary check failed. Student vault access blocked."
        echo "ALERT: Suspicious canary change detected."
        echo "DEFENSE: Student access to the vault has been blocked."
    fi
}
simulate_incident() {
    require_open || return 0
    check_samples || return 0
    if [[ ! -s "$CANARY" ]]; then
        echo "ERROR: Create the clean backup first."
        return 0
    fi
    local answer
    read -r -p "Type DEMO to modify only the sample canary: " answer
    if [[ "$answer" != "DEMO" ]]; then
        echo "Cancelled. No files changed."
        return 0
    fi
    printf '%s\n' "Harmless demonstration change." |
        runuser -u ztv_writer -- tee -a "$VAULT/canary.txt" >/dev/null
    log_event "DEMO: Sample canary deliberately modified."
    echo "Sample canary changed."
    echo "The automatic monitor will detect it if enabled."
    echo "Otherwise, choose option 5 to check now."
}
test_blocked_write() {
    if [[ ! -f "$INCIDENT" ]]; then
        echo "No incident is active. Run the demonstration first."
        return 0
    fi
    if printf '%s\n' "Containment test" |
```

APPENDIX C: BASH SOURCE - 5 OF 8

Bash source listing for ztvault.

```
        runuser -u ztv_writer -- tee -a "$VAULT/assignment.txt" \
        >/dev/null; then
        echo "FAIL: Student could still write."
    else
        echo "PASS: Student cannot write while the vault is locked."
    fi
}
recover_files() {
    if ! verify_backup; then
        echo "Recovery stopped: Backup verification failed."
        return 0
    fi
    local answer stage old
    read -r -p "Restore original files? Type RESTORE: " answer
    if [[ "$answer" != "RESTORE" ]]; then
        echo "Recovery cancelled."
        return 0
    fi
    stage=$(mktemp -d "$BASE/recovery/restore-XXXXXX")
    tar -xzf "$BACKUP" -C "$stage" \
        --no-same-owner --no-same-permissions
    if ! (
        cd "$stage"
        sha256sum --check "$MANIFEST"
        sha256sum --check "$CANARY"
    ); then
        echo "ERROR: Recovered files failed verification."
        echo "Current vault was not replaced."
        return 0
    fi
    chmod 0700 "$VAULT"
    printf '%s\n' "Recovery in progress." > "$INCIDENT"
    old=$(mktemp -d "$BASE/recovery/previous-XXXXXX")
    mv "$VAULT" "$old/data"
    install -d -o root -g ztv_access -m 0700 "$VAULT"
    local name
    for name in assignment.txt marks.txt canary.txt; do
        install -o root -g ztv_access -m 0660 \
            "$stage/$name" "$VAULT/$name"
    done
    (cd "$VAULT"; sha256sum --check "$MANIFEST")
    chmod 2770 "$VAULT"
    rm -f "$INCIDENT"
    log_event "RECOVERY: Original files verified and restored."
    echo "SUCCESS: Original files restored. Student access reopened."
    echo "Previous files preserved in: $old/data"
}
```

APPENDIX C: BASH SOURCE - 6 OF 8

Bash source listing for ztvault.

```
show_log() {
    if [[ -f "$LOG" ]]; then
        tail -n 25 "$LOG"
    else
        echo "No events recorded yet."
    fi
}
enable_monitoring() {
    if [[ ! -s "$CANARY" ]]; then
        echo "ERROR: Create the clean backup first."
        return 0
    fi
    cat > /etc/systemd/system/ztvault-monitor.service <<'EOF'
[Unit]
Description=Student vault canary check
[Service]
Type=oneshot
ExecStart=/usr/local/sbin/ztvault --monitor
UMask=0077
EOF
    cat > /etc/systemd/system/ztvault-monitor.timer <<'EOF'
[Unit]
Description=Check student vault every ten seconds
[Timer]
OnActiveSec=10s
OnUnitActiveSec=10s
AccuracySec=1s
Unit=ztvault-monitor.service
[Install]
WantedBy=timers.target
EOF
    systemctl daemon-reload
    systemctl enable --now ztvault-monitor.timer
    log_event "AUTOMATION: Ten-second security timer enabled."
    echo "SUCCESS: Automatic checks enabled."
    echo "Monitoring runs while this Ubuntu instance is running."
}
show_status() {
    if [[ -f "$INCIDENT" ]]; then
        echo "Vault status: LOCKED"
    else
        echo "Vault status: OPEN"
    fi
    if [[ -f "$BACKUP" ]]; then
        echo "Backup: Present"
    else
        echo "Backup: Not created"
```

APPENDIX C: BASH SOURCE - 7 OF 8

Bash source listing for ztvault.

```
    fi
    if systemctl is-active --quiet ztvault-monitor.timer; then
        echo "Automatic monitoring: ACTIVE"
    else
        echo "Automatic monitoring: INACTIVE"
    fi
}
if [[ "${1:-}" == "--monitor" ]]; then
    flock -n 9 || exit 0
    monitor
    exit 0
fi
if (( $# > 0 )); then
    echo "Usage: sudo ztvault"
    exit 1
fi
while true; do
    echo
    echo "============================================"
    echo "       ZERO-TRUST STUDENT FILE VAULT"
    echo "============================================"
    show_status
    echo "1. View assignment and marks"
    echo "2. Add a sentence to the assignment"
    echo "3. Test access permissions"
    echo "4. Create or verify clean backup"
    echo "5. Check security now"
    echo "6. Simulate a suspicious canary change"
    echo "7. Test write blocking after an incident"
    echo "8. Recover original files"
    echo "9. View security log"
    echo "10. Enable automatic monitoring"
    echo "0. Exit"
    if ! read -r -p "Enter your choice: " choice; then
        break
    fi
    case "$choice" in
        1) action=view_files ;;
        2) action=update_assignment ;;
        3) action=test_permissions ;;
        4) action=create_backup ;;
        5) action=monitor ;;
        6) action=simulate_incident ;;
        7) action=test_blocked_write ;;
        8) action=recover_files ;;
        9) action=show_log ;;
        10) action=enable_monitoring ;;
```

APPENDIX C: BASH SOURCE - 8 OF 8

Bash source listing for ztvault.

```
      0) echo "Application closed."; break ;;
      *) echo "ERROR: Enter a number from 0 to 10."; continue ;;
   esac
   flock 9
   set +e
   (
      set -Eeuo pipefail
      trap 'echo "ERROR: Operation failed. Review the message above." >&2' ERR
      "$action"
   )
   result=$?
   set -e
   flock -u 9
   if (( result != 0 )); then
      echo "Operation did not complete successfully."
   fi
   read -r -p "Press Enter to return to the menu..." || break
done
```

APPENDIX D: IMPLEMENTATION EVIDENCE

D1. Ubuntu environment

The terminal identifies Ubuntu 26.04 LTS, a WSL2 kernel and systemd as process 1.

```
dhars@DHARSHINI:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
dhars@DHARSHINI:~$ uname -r
6.18.33.1-microsoft-standard-WSL2
dhars@DHARSHINI:~$ ps -p 1 -o comm=
systemd
dhars@DHARSHINI:~$
```

D2. Accounts, permissions and filesystem

ztv_writer belongs to ztv_access; ztv_outsider has a separate group. The active vault uses group permissions, protected state is root-only and the containing filesystem is ext4.

```
dhars@DHARSHINI:~$ id ztv_writer
uid=1012(ztv_writer) gid=1016(ztv_access) groups=1016(ztv_access)
dhars@DHARSHINI:~$ id ztv_outsider
uid=1013(ztv_outsider) gid=1017(ztv_outsider) groups=1017(ztv_outsider)
dhars@DHARSHINI:~$ sudo ls -ld /srv/ztvault/data /var/lib/ztvault
[sudo: authenticate] Password:
drwxrws--- 2 root ztv_access 4096 Oct 23 13:20 /srv/ztvault/data
drwx------ 5 root root       4096 Oct 23 13:20 /var/lib/ztvault
dhars@DHARSHINI:~$ findmnt -T /srv/ztvault/data
TARGET SOURCE   FSTYPE OPTIONS
/      /dev/sdd ext4   rw,relatime,discard,errors=remount-ro,data=ordered
dhars@DHARSHINI:~$
```

APPENDIX D: IMPLEMENTATION EVIDENCE

D3. Main menu

sudo ztvault opens the Bash menu. The captured status shows the vault OPEN, a backup present and automatic monitoring ACTIVE.

```
dhars@DHARSHINI:~$ sudo ztvault

========================================
       ZERO-TRUST STUDENT FILE VAULT
========================================
Vault status: OPEN
Backup: Present
Automatic monitoring: ACTIVE

1. View assignment and marks
2. Add a sentence to the assignment
3. Test access permissions
4. Create or verify clean backup
5. Check security now
6. Simulate a suspicious canary change
7. Test write blocking after an incident
8. Recover original files
9. View security log
10. Enable automatic monitoring
0. Exit

Enter your choice:
```

D4a. User input and saved output - input / first result

Option 2 accepts a sentence and confirms the update. Option 1 then displays the same sentence in assignment.txt together with the sample marks.

```
Enter your choice: 2
Enter one sentence to add to the assignment: I completed my Linux assignment.
SUCCESS: Assignment updated.
Press Enter to return to the menu...
```

APPENDIX D: IMPLEMENTATION EVIDENCE

D4b. User input and saved output - verification / second result

User input processed and saved in the assignment.

```
Enter your choice: 1
----- ASSIGNMENT -----
STUDENT ASSIGNMENT
Student: Demo Student
Subject: Linux System Administration
Topic: File Permissions
Answer: Linux permissions control who can read, write, or execute a file.

Status: Completed
I completed my Linux assignment.

----- SAMPLE MARKS -----
SAMPLE MARKS
Linux: 90
Python: 95
Mathematics: 88
Press Enter to return to the menu...
```

D5. Access tests

The student successfully reads the assignment. Both outsider reading and student access to protected backups are denied, with PASS messages for the expected restrictions.

```
Enter your choice: 3
----- STUDENT READ TEST -----
STUDENT ASSIGNMENT
Student: Demo Student
Subject: Linux System Administration
Topic: File Permissions
Answer: Linux permissions control who can read, write, or execute a file.

Status: Completed
I completed my Linux assignment.
PASS: Student can read the assignment.

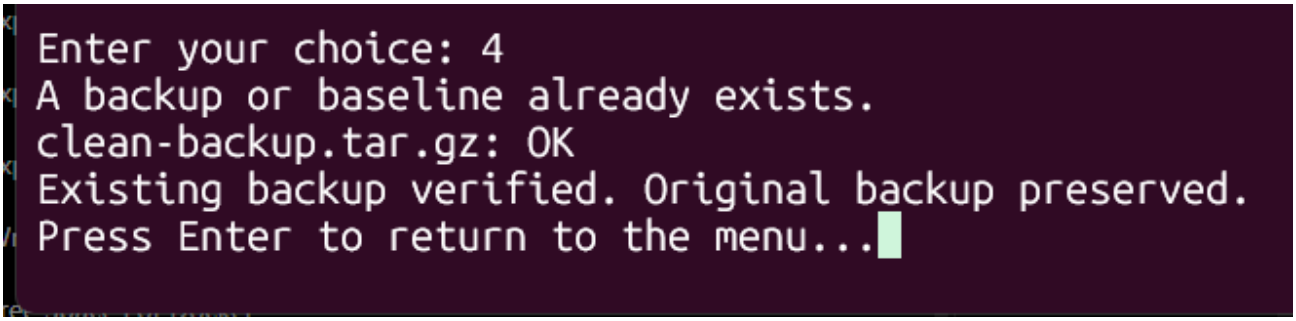
----- OUTSIDER READ TEST -----
cat: /srv/ztvault/data/assignment.txt: Permission denied
PASS: Outsider access denied.

----- STUDENT BACKUP ACCESS TEST -----
ls: cannot open file '/var/lib/ztvault/backups': Permission denied
PASS: Protected backup access denied.
Press Enter to return to the menu...
```

APPENDIX D: IMPLEMENTATION EVIDENCE

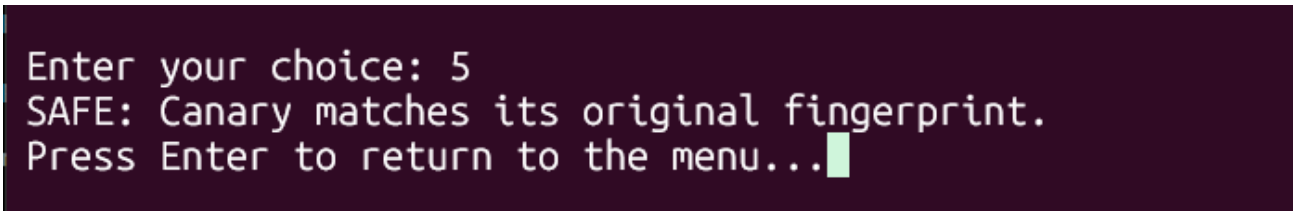
D6. Existing backup verification

Option 4 finds an existing clean backup. Its archive checksum returns OK, and the script preserves the original. This screenshot proves verification, not a new backup creation in this run.



D7. Normal integrity check

After the legitimate assignment edit, option 5 reports SAFE. The edit did not change the canary fingerprint.



APPENDIX D: IMPLEMENTATION EVIDENCE

D8a. Automatic monitoring - input / first result

Option 10 confirms automatic checks. The timer status shows the unit enabled and active (waiting), with its next trigger and linked service.

```
Enter your choice: 10
SUCCESS: Automatic checks enabled.
Monitoring runs while this Ubuntu instance is running.
Press Enter to return to the menu...
```

D8b. Automatic monitoring - verification / second result

Enabled and active systemd monitoring timer.

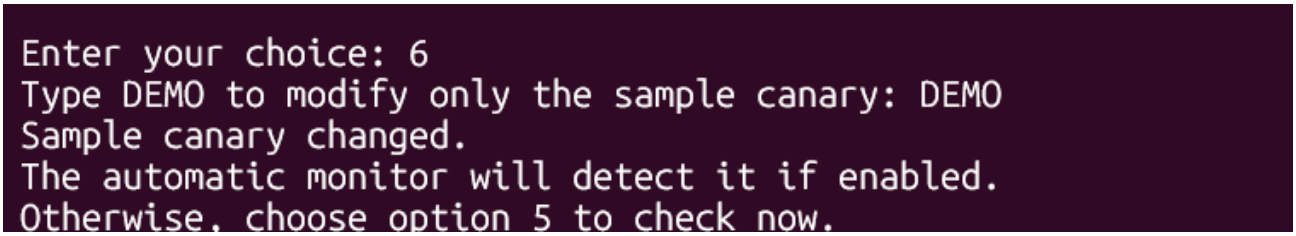
```
dhars@DHARSHINI:~$ systemctl status ztvault-monitor.timer --no-pager
● ztvault-monitor.timer - Check student vault every ten seconds
     Loaded: loaded (/etc/systemd/system/ztvault-monitor.timer; enabled; preset: enabled)
     Active: active (waiting) since Fri 2026-10-23 09:05:21 UTC; 4h 31min ago
 Invocation: 78e5e3ab342b4e268038b2c629b6628e
    Trigger: Fri 2026-10-23 13:37:00 UTC; 2s left
   Triggers: ● ztvault-monitor.service

Oct 23 09:05:21 DHARSHINI systemd[1]: Started ztvault-monitor.timer - Check student vault every ten seconds.
dhars@DHARSHINI:~$
```

APPENDIX D: IMPLEMENTATION EVIDENCE

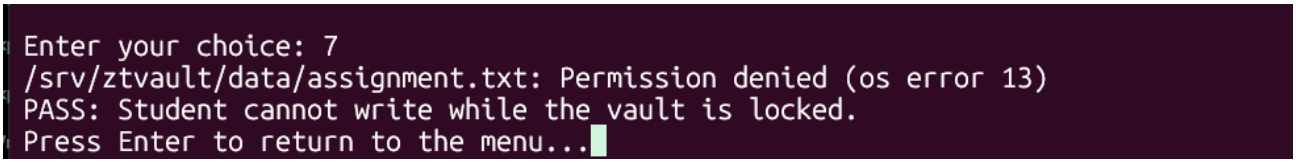
D9. Safe incident simulation

Option 6 accepts the exact word DEMO and confirms that the sample canary changed. The operation is a harmless test modification, not malware execution.



D10a. Automatic detection and containment - input / first result

The student write attempt is denied. The service journal records ALERT and DEFENSE at 13:37:34 UTC, followed by repeated LOCKED checks. This supports timer-driven detection and containment.



APPENDIX D: IMPLEMENTATION EVIDENCE

D10b. Automatic detection and containment - verification / second result

Student write denial and service-journal evidence of detection.

```
dhars@DHARSHINI:~$ sudo journalctl -u ztvault-monitor.service -n 20 --no-pager
[sudo: authenticate] Password:
Oct 23 13:37:34 DHARSHINI ztvault[6133]: ALERT: Suspicious canary change detected.
Oct 23 13:37:34 DHARSHINI ztvault[6133]: DEFENSE: Student access to the vault has been blocked.
Oct 23 13:37:34 DHARSHINI systemd[1]: ztvault-monitor.service: Deactivated successfully.
Oct 23 13:37:34 DHARSHINI systemd[1]: Finished ztvault-monitor.service - Student vault canary check.
Oct 23 13:37:45 DHARSHINI systemd[1]: Starting ztvault-monitor.service - Student vault canary check...
Oct 23 13:37:45 DHARSHINI ztvault[6139]: LOCKED: Incident active. Recovery is required.
Oct 23 13:37:45 DHARSHINI systemd[1]: ztvault-monitor.service: Deactivated successfully.
Oct 23 13:37:45 DHARSHINI systemd[1]: Finished ztvault-monitor.service - Student vault canary check.
Oct 23 13:37:56 DHARSHINI systemd[1]: Starting ztvault-monitor.service - Student vault canary check...
Oct 23 13:37:56 DHARSHINI ztvault[6152]: LOCKED: Incident active. Recovery is required.
Oct 23 13:37:56 DHARSHINI systemd[1]: ztvault-monitor.service: Deactivated successfully.
Oct 23 13:37:56 DHARSHINI systemd[1]: Finished ztvault-monitor.service - Student vault canary check.
Oct 23 13:38:07 DHARSHINI systemd[1]: Starting ztvault-monitor.service - Student vault canary check...
Oct 23 13:38:07 DHARSHINI ztvault[6156]: LOCKED: Incident active. Recovery is required.
Oct 23 13:38:07 DHARSHINI systemd[1]: ztvault-monitor.service: Deactivated successfully.
Oct 23 13:38:07 DHARSHINI systemd[1]: Finished ztvault-monitor.service - Student vault canary check.
Oct 23 13:38:18 DHARSHINI systemd[1]: Starting ztvault-monitor.service - Student vault canary check...
Oct 23 13:38:18 DHARSHINI ztvault[6178]: LOCKED: Incident active. Recovery is required.
Oct 23 13:38:18 DHARSHINI systemd[1]: ztvault-monitor.service: Deactivated successfully.
Oct 23 13:38:18 DHARSHINI systemd[1]: Finished ztvault-monitor.service - Student vault canary check.
dhars@DHARSHINI:~$
```

D11. Security event log

The log records DEMO at 13:37:30 and ALERT at 13:37:34, along with earlier updates, automation and recovery events. The observed detection delay in this run was four seconds.

```
Enter your choice: 9
2026-10-23 09:04:49 | UPDATE: Student added a sentence to the assignment.
2026-10-23 09:05:21 | AUTOMATION: Ten-second security timer enabled.
2026-10-23 09:05:43 | DEMO: Sample canary deliberately modified.
2026-10-23 09:05:53 | ALERT: Canary check failed. Student vault access blocked.
2026-10-23 13:20:48 | RECOVERY: Original files verified and restored.
2026-10-23 13:34:51 | UPDATE: Student added a sentence to the assignment.
2026-10-23 13:36:24 | AUTOMATION: Ten-second security timer enabled.
2026-10-23 13:37:30 | DEMO: Sample canary deliberately modified.
2026-10-23 13:37:34 | ALERT: Canary check failed. Student vault access blocked.
Press Enter to return to the menu...
```

APPENDIX D: IMPLEMENTATION EVIDENCE

D12. Verified recovery

Option 8 accepts RESTORE. The archive and sample-file checks return OK. The script reports successful restoration, reopened student access and the protected location of the previous files.

```
Enter your choice: 8
clean-backup.tar.gz: OK
Restore original files? Type RESTORE: RESTORE
assignment.txt: OK
marks.txt: OK
canary.txt: OK
canary.txt: OK
assignment.txt: OK
marks.txt: OK
canary.txt: OK
SUCCESS: Original files restored. Student access reopened.
Previous files preserved in: /var/lib/ztvault/recovery/previous-vGQbEB/data
Press Enter to return to the menu...
```

D13a. Restored original and SAFE state - input / first result

The restored assignment no longer contains the sentence added in D4. A subsequent option-5 check reports SAFE, confirming that the restored canary matches the baseline.

```
Enter your choice: 1
----- ASSIGNMENT -----
STUDENT ASSIGNMENT
Student: Demo Student
Subject: Linux System Administration
Topic: File Permissions
Answer: Linux permissions control who can read, write, or execute a file.

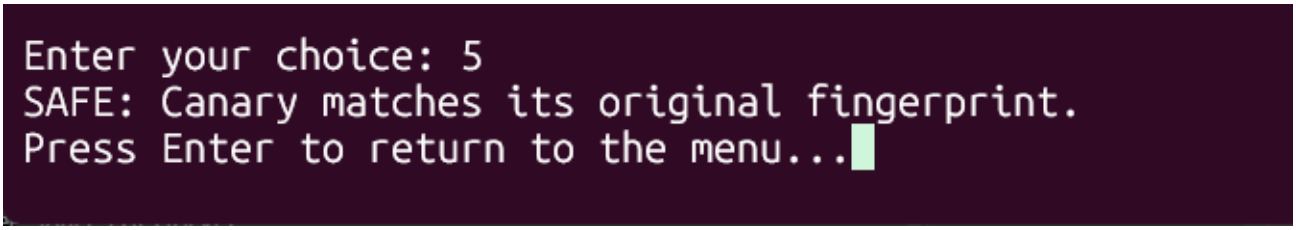
Status: Completed

----- SAMPLE MARKS -----
SAMPLE MARKS
Linux: 90
Python: 95
Mathematics: 88
Press Enter to return to the menu...
```

APPENDIX D: IMPLEMENTATION EVIDENCE

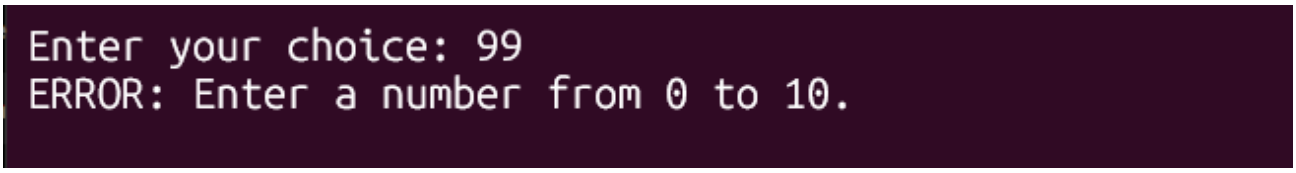
D13b. Restored original and SAFE state - verification / second result

Original documents restored and canary verification passing.



D14a. Invalid-input handling - input / first result

Choice 99 produces a menu-range error. An empty sentence produces a separate validation error. The messages guide the user without accepting invalid input.



D14b. Invalid-input handling - verification / second result

Invalid menu choice and empty assignment input rejected.

